

Free Text and NLP in Data Science

1. Introduction to Free Text and NLP

- Definition and scope of Free Text and Natural Language Processing.
- Importance of NLP in data science.

2. Basics of Text Processing

- Text Cleaning
- Tokenization
- Stemming and Lemmatization
- Normalization Techniques

3. Feature Extraction from Text

- **Bag of Words (BoW):**
 - Concept and implementation.
 - Pros and cons of using BoW.
- **TF-IDF (Term Frequency-Inverse Document Frequency):**
 - Understanding term importance and its calculation.
 - Implementing TF-IDF in Python.
- **Word Embeddings:**
 - Introduction to word embeddings (Word2Vec, GloVe).
 - Using pre-trained embeddings in NLP tasks.
- **Language models and N-grams**

4. Advanced NLP Techniques

- **Recurrent Neural Networks (RNNs):**
 - Overview of RNNs in NLP.
 - Introduction to LSTM and GRU networks.
- **Transformer Models:**

- Introduction to Transformer architecture.
- Overview of BERT, GPT, and other modern models.
- **Transfer Learning in NLP:**
 - Fine-tuning pre-trained models for specific tasks.

5. NLP Project Pipeline

- **Data Collection:**
 - Sources for NLP datasets (Kaggle, UCI, public APIs).
 - Ethical considerations in text data collection.
- **Exploratory Data Analysis (EDA) for Text:**
 - Visualizing text data distributions.
 - Common EDA techniques for text data.
- **Model Building:**
 - Selecting the right model for the task (classification, generation, etc.).
 - Training and evaluating NLP models.
- **Model Evaluation Metrics:**
 - Accuracy, precision, recall, F1-score, BLEU, ROUGE, etc.

6. Future Trends in NLP

- **Emerging Technologies:**
 - Low-resource NLP, multilingual models, and zero-shot learning.
- **Automated Text Generation:**
 - Exploring advancements in generative models like ChatGPT.
- **Integration of NLP with Other AI Domains:**
 - Combining NLP with computer vision, robotics, and more.

7. Summary

- **Recap of Key Concepts:**
 - Reviewing the main takeaways from the course.
- **Further Learning Resources:**

- Books, courses, and online resources for deeper exploration of NLP.

✓ Free Text and NLP

This set of notes focuses on processing free text data. In addition to the more structured data, free text makes up one of the most common types of "widely available" data: web pages, unstructured "comment" fields in many relational databases, and many other easily-obtained large sources of data naturally come in free text form. The notable difference, of course, is that unlike the data types we have discussed before, free text lacks the "easily extractable" structure inherent in the previous types of data we have considered.

This is not to say, of course, that free text data lacks structure. Just the opposite: by its very definition free text usually needs to have meaning to the people who are reading that data. But the task of actually understanding this structure is a problem that is still beyond the scope of "generic" data science tools, because it often involves human-level intelligence (or at least bringing to bear an enormous amount of external context) to understand the meaning behind the text.

✓ 1. Introduction to Free Text and NLP

As mentioned above, the goal that we will consider here of using free text in data science is to extract some meaningful information from the text, *without* any deep understanding of its meaning. The reason for this is simple: extracting deep understanding from text is hard.

✓ Natural language processing/understanding

The general field of Natural Language Processing (some might differentiate between NLP and "natural language understanding", but for our purposes here you can think of these synonymously) looks to truly understand the structure behind free text, e.g. perform tasks like parse the sentences grammatically, describe the general entities and properties that the text is referring to, etc. But it is easy to come up with examples of why this can be hard. For example, consider the following example, known

examples of why this can be hard. For example, consider the following example, known as a Winograd schema:

The city councilmen refused the demonstrators a permit because they [feared/advocated] violence.

If you read the sentence:

The city councilmen refused the demonstrators a permit because they feared violence.

then it is fairly obviously that the "they" here refers to the city councilmen; fearing violence would not be a reason for denying someone a permit, so clearly it is the councilmen that fear violence. On the other hand, in the sentence

The city councilmen refused the demonstrators a permit because they advocated violence.

the "they" term clearly applies to the demonstrators (the city councilmen presumably would not advocate for violence, let alone deny someone a permit because they, the councilmen, were advocating violence). The point of all this (and this was the original point of these examples, which were originally proposed by Terry Winograd in 1972) is that when we do something "simple", like parse a sentence, we bring to bear an enormous amount of outside knowledge and context to this action. Unlike say, XML documents, there is no internally-specified format that makes language unambiguous; it could be unambiguous only because of external context and knowledge, or it could even be completely ambiguous. Or in a different vein, there can be grammatically-incorrect sentences that still have clear syntactic meaning, and there can be grammatically-correct sentences that are meaningful.

Terminology:

Before we begin, a quick note on terminology.

- In these notes "document" will mean an individual group of free text data (this could be an actual document or a text field in a database).
- "Words" or "terms" refer to individual tokens separated by whitespace, and additionally also refers to punctuation (so we will often separate punctuation explicitly from the surrounding words).
- "Corpus" refers to a collection of documents, and we will sometimes refer to the set of all *unique* words/tokens in the corpus as the "vocabulary".

```
!pip install nltk
```

```
Requirement already satisfied: nltk in /usr/local/lib/python3.10/dist-  
Requirement already satisfied: click in /usr/local/lib/python3.10/dist-  
Requirement already satisfied: joblib in /usr/local/lib/python3.10/dis-  
Requirement already satisfied: regex>=2021.8.3 in /usr/local/lib/pytho-  
Requirement already satisfied: tqdm in /usr/local/lib/python3.10/dist-
```

✓ 2. Basics of Text Processing

Text processing is a crucial step in preparing textual data for Natural Language Processing (NLP) tasks. It involves transforming raw text into a structured format that can be used effectively by machine learning models.

✓ 2.1. Text Cleaning

Text cleaning involves removing or modifying unwanted elements from the text that do not add value to the model's learning process. Common cleaning steps include:

- **Lowercasing:** Converts all characters in the text to lowercase to ensure uniformity.
- **Removing Punctuation:** Eliminates punctuation marks that do not contribute to the meaning of the text.
- **Removing Stopwords:** Removes commonly used words (e.g., 'and', 'is', 'in') that do not carry significant meaning.
- **Removing Special Characters and Numbers:** Eliminates special characters and numbers that are often irrelevant for textual analysis.

Example:

```
import re
import string
from nltk.corpus import stopwords
from nltk.tokenize import word_tokenize
import nltk

# Download the necessary resources
nltk.download('stopwords')
nltk.download('punkt')

def clean_text(text):
    """
    Cleans the input text by removing punctuation, stopwords, special

    Parameters:
    text (str): The input text to clean.

    Returns:
    str: The cleaned text.
    """
    # Convert text to lowercase
    text = text.lower()

    # Remove punctuation
    text = text.translate(str.maketrans('', '', string.punctuation))
```

```
# Remove numbers
text = re.sub(r'\d+', '', text)

# Remove special characters
text = re.sub(r'[^a-z\s]', '', text)

# Remove stopwords
stop_words = set(stopwords.words('english'))
text = ' '.join(word for word in word_tokenize(text) if word not in stop_words)

return text

sample_text = "Hello! This is an example text with numbers 123 and punctuation."
cleaned_text = clean_text(sample_text)
print(cleaned_text) # Output: hello example text numbers
```

```
[nltk_data] Downloading package stopwords to /root/nltk_data...
[nltk_data]   Package stopwords is already up-to-date!
[nltk_data] Downloading package punkt to /root/nltk_data...
[nltk_data]   Unzipping tokenizers/punkt.zip.
hello example text numbers punctuation
```

✓ 2.2. Tokenization

Tokenization is the process of splitting text into smaller units, typically words or sentences. It helps in breaking down text data into manageable pieces for further processing.

- **Word Tokenization:** Splits the text into individual words.
- **Sentence Tokenization:** Splits the text into sentences.

Example:

```

from nltk.tokenize import word_tokenize, sent_tokenize

def tokenize_text(text):
    """
    Tokenizes the input text into words and sentences.

    Parameters:
    text (str): The input text to tokenize.

    Returns:
    list: A list of words and a list of sentences.
    """
    # Word tokenization
    words = word_tokenize(text)

    # Sentence tokenization
    sentences = sent_tokenize(text)

    return words, sentences

sample_text = "Natural Language Processing (NLP) is exciting! It enables"
words, sentences = tokenize_text(sample_text)
print("Words:", words)
print("Sentences:", sentences)

```

Words: ['Natural', 'Language', 'Processing', '(', 'NLP', ')', 'is', 'e']
 Sentences: ['Natural Language Processing (NLP) is exciting!', 'It enab']

✓ 2.3. Stemming and Lemmatization

- **Stemming:** Reduces words to their base or root form by stripping suffixes (e.g., 'running' becomes 'run'). Stemming may result in non-dictionary words.
- **Lemmatization:** Reduces words to their base or dictionary form (lemma) while considering the context (e.g., 'running' becomes 'run', but 'better' becomes 'good').

Example:

```

from nltk.stem import PorterStemmer, WordNetLemmatizer
from nltk.tokenize import word_tokenize
import nltk

```

```
# Download the necessary resources
nltk.download('punkt')
nltk.download('wordnet')

def apply_stemming(text):
    """
    Applies stemming to the input text.

    Parameters:
    text (str): The input text to stem.

    Returns:
    list: A list of stemmed words.
    """
    # Initialize the stemmer
    stemmer = PorterStemmer()

    # Tokenize text into words
    words = word_tokenize(text)

    # Apply stemming
    stemmed_words = [stemmer.stem(word) for word in words]

    return stemmed_words

def apply_lemmatization(text):
    """
    Applies lemmatization to the input text.

    Parameters:
    text (str): The input text to lemmatize.

    Returns:
    list: A list of lemmatized words.
    """
    # Initialize the lemmatizer
    lemmatizer = WordNetLemmatizer()

    # Tokenize text into words
    words = word_tokenize(text)

    # Apply lemmatization
    lemmatized_words = [lemmatizer.lemmatize(word) for word in words]
```

```

    return lemmatized_words

sample_text = "The children are playing with their toys."
stemmed_words = apply_stemming(sample_text)
lemmatized_words = apply_lemmatization(sample_text)
print("Stemmed Words:", stemmed_words) # Output: ['the', 'children', 'are', 'play', 'with', 'their', 'toys']
print("Lemmatized Words:", lemmatized_words) # Output: ['the', 'child', 'are', 'play', 'with', 'their', 'toy']

```

```
[nltk_data] Downloading package punkt to /root/nltk_data...
[nltk_data]   Package punkt is already up-to-date!
[nltk_data] Downloading package wordnet to /root/nltk_data...
Stemmed Words: ['the', 'children', 'are', 'play', 'with', 'their', 'to']
Lemmatized Words: ['The', 'child', 'are', 'playing', 'with', 'their',
```

2.4. Normalization Techniques

Normalization techniques standardize text data by reducing it to a uniform format. This can include:

- **Handling Synonyms:** Converting synonyms to a common representation (e.g., 'car', 'automobile' both become 'car').
- **Handling Abbreviations:** Expanding abbreviations to their full forms (e.g., 'u' becomes 'you').
- **Spelling Correction:** Correcting misspelled words.

Example:

```
!pip install autocorrect
```

```
Collecting autocorrect
  Downloading autocorrect-2.6.1.tar.gz (622 kB)
    

622.8/622.8 kB 12.0 MB/s


  Preparing metadata (setup.py) ... done
Building wheels for collected packages: autocorrect
  Building wheel for autocorrect (setup.py) ... done
  Created wheel for autocorrect: filename=autocorrect-2.6.1-py3-none-any.whl
  Stored in directory: /root/.cache/pip/wheels/b5/7b/6d/b76b29ce11ff8e...
Successfully built autocorrect
Installing collected packages: autocorrect
Successfully installed autocorrect-2.6.1
```

```
from nltk.tokenize import word_tokenize
```

```

from autocorrect import Speller

# Download necessary resources
nltk.download('punkt')

def normalize_text(text):
    """
    Normalizes the input text by expanding abbreviations, correcting spelling,
    and handling special characters appropriately.

    Parameters:
    text (str): The input text to normalize.

    Returns:
    str: The normalized text.
    """
    # Initialize spell checker
    spell = Speller(lang='en')

    # Define a dictionary for common abbreviations
    normalization_dict = {
        "u": "you",
        "ur": "your",
        "r": "are",
        "pls": "please",
        "pos": "please",
        "thx": "thanks"
    }

    # Tokenize text into words
    words = word_tokenize(text.lower())

    # Handle abbreviations, spelling corrections, and remove special characters
    normalized_words = [normalization_dict.get(word, spell(word)) for word in words]

    # Reconstruct the text
    normalized_text = ' '.join(normalized_words)

    # Clean special characters after spelling corrections
    normalized_text = normalized_text.replace('!', '').replace('.', '')

    return normalized_text

# Example usage
sample_text = "U r the best! Pls help me with this assignment. Thx!"
normalized_text = normalize_text(sample_text)

```

```
normalized_text = normalize_text(sample_text)
print(normalized_text) # Output: you are the best please help me with
```

```
you are the best please help me with this assignment thanks
[nltk_data] Downloading package punkt to /root/nltk_data...
[nltk_data] Package punkt is already up-to-date!
```

Summary:

- **Text Cleaning:** Involves removing unnecessary elements such as punctuation, stopwords, and special characters.
- **Tokenization:** Splits text into individual words or sentences for further analysis.
- **Stemming and Lemmatization:** Reduces words to their root forms, with lemmatization providing contextually correct lemmas.
- **Normalization Techniques:** Standardizes text by handling synonyms, abbreviations, and correcting spelling errors.

These foundational text processing steps are essential in preparing text data for more advanced NLP tasks.

✓ 3. Feature Extraction from Text

Fortunately, for many data science tasks, we can still extract considerable information from text *while only understanding it at an extremely rudimentary level*. Consider the following two reviews for the same movie:

... truly, a stunning exercise in large-scale filmmaking; a beautifully-assembled picture in which Abrams combines a magnificent cast with a marvelous flair for big-screen, sci-fi storytelling.

and

It's loud and full of vim -- but a little hollow and heartless.

Understanding that the first review is positive, while the second review is negative, doesn't take any deep understand of the language itself, it can be done by simple keyword lookup: "stunning" and "marvelous" are associated with a positive review, while "hollow" and "heartless" are associated with negative reviews. Now, of course it's possible to use more complex language to signify a positive review while using some "negative" words, with statements like, "not at all boring". But people don't usually write exclusively in this manner (doing so would be "not at all clear"), so that the general sentiment of text can still come through very easily even with a few instances where the words themselves can throw you off.

With this as context, these notes cover three different examples of methods for extracting some information from free text, without truly understanding much of the language itself. These methods are becoming standard tools in data science, and have the advantage of being relatively easy to set up and begin processing with. These methods are:

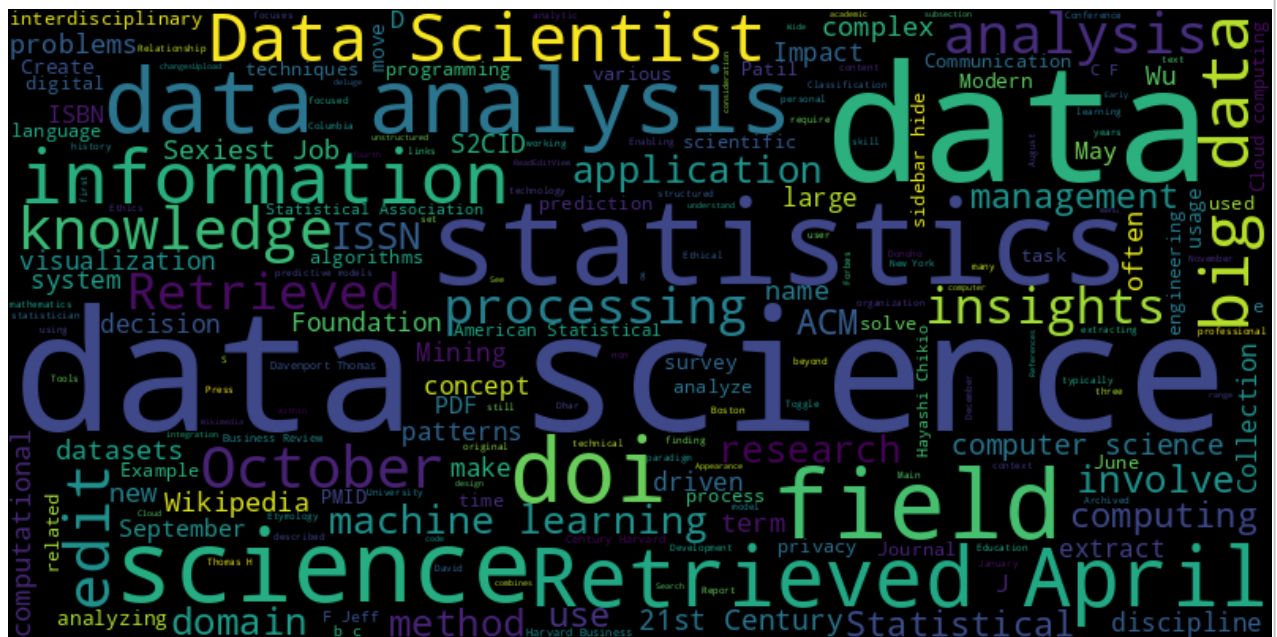
1. Bag of words models and TDIDF vectors
2. Word embeddings and word2vec
3. N-gram language models

3.1. Bag of words models and TFIDF

The bag of words model is by far the most common means of representing documents in data science. Under this model, a document is described solely by the set of words (and possibly their counts) that make up the document. All information about the actual ordering of the words is ignored. It is essentially the so-called "word cloud" view of a document.

```
import wordcloud
from bs4 import BeautifulSoup
import requests
import re

response = requests.get("https://en.wikipedia.org/wiki/Data_science")
root = BeautifulSoup(response.content, "lxml")
from wordcloud import WordCloud
wc = WordCloud(width=800,height=400).generate(re.sub(r"\s+"," ", root.
wc.to_image()
```



Despite the obvious information we are throwing away with this representation, it send to work surprisingly well in practice, for the precise reason we mentioned above, that the general "gist" of many documents can be obtained with only looking at the presence/absence of words in the text.

In these notes, we'll cover a simple example of creating so-called TFIDF vectors, which represent the documents via a (weighted) bag of words model. We will then then using these to compute the similarity between documents. This technique is a common approach for applications like document retrieval and or search, though in the homework you will also experiment using TFIDF vectors as input to a machine learning algorithm.

✓ Term frequency

In our example, let's begin with our corpus that contains the following three documents:

```
documents = ["the goal of this sprint is to explain the basics of free  
             "the bag of words model is one such approach",  
             "text processing via bag of words"]
```

In this setting, we can represent the documents using a *term frequency* matrix, and $m \times n$ matrix where m denotes the number of documents, and n denotes the vocabulary size (i.e., the number of unique words across all documents). To see (the naive way of) how to construct this list, let's first consider a simple way to get a list of all unique words across all documents. In general there is no need to actually sort the list of words, but we will do so for simplicity here. It's a good idea to also generate a dictionary that maps words to their index in this list, as we'll frequently want to look up the index corresponding to a word.

```
document_words = [doc.split() for doc in documents]
vocab = sorted(set(sum(document_words, [])))
vocab_dict = {k:i for i,k in enumerate(vocab)}
print(vocab, "\n")
print(vocab_dict, "\n")
```

```
['approach', 'bag', 'basics', 'explain', 'free', 'goal', 'is', 'model'
{'approach': 0, 'bag': 1, 'basics': 2, 'explain': 3, 'free': 4, 'goal':
```

Now let's construct a matrix that contains word counts (term frequencies) for all the documents. We'll also refer to the term frequency of the j th word in the i th library as $tf_{i,j}$.

```
import numpy as np
X_tf = np.zeros((len(documents), len(vocab)), dtype=int)
for i,doc in enumerate(document_words):
    for word in doc:
        X_tf[i, vocab_dict[word]] += 1
print(X_tf)
```

```
[[0 0 1 1 1 1 1 0 2 0 1 1 0 1 2 1 1 0 0]
 [1 1 0 0 0 0 1 1 1 1 0 0 1 0 1 0 0 0 1]
 [0 1 0 0 0 0 0 0 1 0 1 0 0 1 0 0 0 1 1]]
```

As mentioned above, each of the *rows* in this matrix corresponds to one of the three documents above, and each column corresponds to one of the 19 words. Importantly, note that in practice you will want to create term frequency matrices directly in sparse format, because the term frequency matrix is itself typically sparse (when there are a large number of documents, many words will only be contained in a small number of the document).

The term *sparse* refers to datasets where many of the elements are zero or missing. For example, in a large matrix used for machine learning, if most of the entries are zero, the matrix is considered sparse. This often occurs in fields like natural language processing (NLP), where a document-term matrix may have many zeros because not every word appears in every document.

In this case, we had the entry in our term frequency matrix just correspond to the number of occurrences of that term. But there are other possibilities as well:

- The entry can be binary: 1 if the term occurs (any number of times), and 0 otherwise. This somewhat mitigates the significance of common words that may occur very frequently.
- A nonlinear scaling, e.g. $\log(1 + \text{tf}_{i,j})$, which lies somewhere between the binary case and the raw counts.
- A scaled version of term frequencies, e.g., scaling by the maximum term frequency in the document, $\text{tf}_{i,j} / \max_k \text{tf}_{i,k}$. (Note that this won't affect the actual similarity scores we'll discuss next, because we will ultimately scale each document but it affects the term frequency matrix itself, of course).

▾ Inverse document frequency

An obvious issue with using normal term frequency counts to represent a document is that the document's vector (and the resulting similarities we will consider) will often be "dominated" by very common words, for example: "of", "the", "is", in the preceeding example documents. This issue can be mitigated to some extent by excluding so-called "stop words" (common English words like "the", "a", "of" that aren't considered relevant to the particular documents) from the term frequency matrix. But this still ignores the case where a word that may not be a generic stop word still happens to occur in a very large number of documents. Intuitively, we expect that the most "important" words in a document are precisely those that only occur in some relatively small number of documents, so that we want to discount the weight of very frequently-occurring terms.

This can be accomplished via the inverse document frequency weight for words. Just as with term frequencies, there are some different weightings of this term, but the most common formulation is

$$\text{idf}_j = \log\left(\frac{\# \text{ documents}}{\# \text{ documents with word } j}\right).$$

As an example, if the word is contained in every document, then the inverse document frequency weight will be zero (log of one). In contrast, if a word occurs in only one document, its inverse document frequency will be $\log(\# \text{ documents})$.

Note that inverse document frequency is a *per word* term, as opposed to term frequency, which is *per word and document*. We can compute inverse document frequency for our data set as follows, which mainly just requires counting how many documents contain each word.

```
idf = np.log(X_tf.shape[0]/X_tf.astype(bool).sum(axis=0))
print(idf)
```

```
[1.09861229  0.40546511  1.09861229  1.09861229  1.09861229  1.09861229
 0.40546511  1.09861229  0.          1.09861229  0.40546511  1.09861229
 1.09861229  0.40546511  0.40546511  1.09861229  1.09861229  1.09861229
 0.40546511]
```

▼ TFIDF

The *term frequency inverse document frequency (TFIDF)* combination simply scales the columns of the term frequency matrix by their inverse document frequency. In doing so, we still have an effective bag of words representation of each document, but we do so with the weighting implied by the inverse document frequency: discounting words that occur very frequently, and increasing the weight of less frequent terms.

```
X_tfidf = X_tf * idf
print(X_tfidf)
```

```
[[0.          0.          1.09861229 1.09861229 1.09861229 1.09861229
  0.40546511 0.          0.          0.          0.40546511 1.09861229
  0.          0.40546511 0.81093022 1.09861229 1.09861229 0.
  0.          ]
 [1.09861229 0.40546511 0.          0.          0.          0.
  0.40546511 1.09861229 0.          1.09861229 0.          0.
  1.09861229 0.          0.40546511 0.          0.          0.
  0.40546511]
 [0.          0.40546511 0.          0.          0.          0.
  0.          0.          0.          0.          0.40546511 0.
  0.          0.40546511 0.          0.          0.          1.09861229
  0.40546511]]
```

✓ Cosine similarity

Given a TFIDF (or just term frequency) matrix, one of the more common questions to address is to compute similarity between multiple documents in the corpus. The common metric for doing so is to compute the cosine similarity between two different documents. This is simply a normalized inner product between the vectors describing each document. Specifically,

$$\text{CosineSimilarity}(x, y) = \frac{x^T y}{\|x\|_2 \cdot \|y\|_2}.$$

The cosine similarity is a number between zero (meaning the two documents share no terms in common) and one (meaning the two documents have the exact same term frequency or TFIDF representation). In fact, the cosine similarity is exactly the converse of the squared Euclidean distance between the normalized document vectors; formally, for $\tilde{x} = x/\|x\|_2$ and $\tilde{y} = y/\|y\|_2$,

$$\begin{aligned} \frac{1}{2} \|\tilde{x} - \tilde{y}\|_2^2 &= \frac{1}{2} (\tilde{x} - \tilde{y})^T (\tilde{x} - \tilde{y}) \\ &= \frac{1}{2} (\tilde{x}^T \tilde{x} - 2\tilde{x}^T \tilde{y} + \tilde{y}^T \tilde{y}) \\ &= \frac{1}{2} (1 - 2\tilde{x}^T \tilde{y} + 1) \\ &= 1 - \text{CosineSimilarity}(x, y). \end{aligned}$$

We can compute cosine similarity between the TFIDF vectors in our corpus as follows.

```
X_tfidf_norm = X_tfidf / np.linalg.norm(X_tfidf, axis=1)[:,None]
M = X_tfidf_norm @ X_tfidf_norm.T
print(M)
```

```
[[1.          0.06796739 0.07771876]
 [0.06796739 1.          0.10281225]
 [0.07771876 0.10281225 1.          ]]
```

We could also look at the cosine similarity with the ordinary term frequency matrix.

```
X_tf_norm = X_tf / np.linalg.norm(X_tf, axis=1)[:,None]
M = X_tf_norm @ X_tf_norm.T
print(M)
```

```
[[1.          0.3928371  0.38490018]
 [0.3928371  1.          0.40824829]
 [0.38490018 0.40824829 1.          ]]
```

As expected, using the term frequency matrix results in substantially higher scores: the inclusion of words that occur across many of the documents add positive terms to the inner product between the document vectors, resulting in higher similarity scores. Note, however, that the distances are typically *all* scaled up, meaning that the relative distances are not necessarily any more informative than the TFIDF similarities (indeed, it is typically less informative, since the scores are inflated by the "random" occurrence of multiple high-frequency words).

✓ 3.2. Word embeddings and word2vec

One of the downsides to term frequency models (even as far as bag-of-words representations are concerned), is there is no notion of similarity between words. But in reality, we do associate some notion of similarity between different words. For example, consider the following corpus.

```
documents = [
    "pittsburgh has some excellent new restaurants",
    "boston is a city with great cuisine",
    "postgresql is a relational database management system"
]
```

In a term frequency model (one that doesn't throw out stop words, but it is naturally possible to construct similar examples even with less common words), the second and third documents have the highest similarity (due to them both having the words "is" and "a"). Conversely, there is no exact word overlap between the first sentence and either other sentence, so the similarity will be zero. Yet this doesn't quite seem satisfactory, because there is an obvious sense in which the first two sentences are more similar than the third in the above example: they are both about food in cities, whereas the third sentence seems entirely unrelated.

To get started with word embeddings, we'll note that words in the term frequency model can also be considered as being represented by vectors; in particular, any word is represented by a "one-hot" vector that has a zero in all the coordinates except a one in the coordinate specified by the word. For example, let's form the vocabulary for the corpus above.

```
document_words = [doc.split() for doc in documents]
vocab = sorted(set(sum(document_words, [])))
vocab_dict = {k:i for i,k in enumerate(vocab)}
print(vocab_dict, "\n")
```

```
{'a': 0, 'boston': 1, 'city': 2, 'cuisine': 3, 'database': 4, 'excellen
```

In this case the vector for "pittsburgh" would just be an all-zeros vector but with a one in the 11th position. We'll denote this vector as $e_{\text{pittsburgh}}$, as the "e" notation often stands for the unit basis in linear algebra (exactly this same idea of a vector that is all zeros with a single one in a particular entry).

```
e_pittsburgh = np.zeros(len(vocab))
e_pittsburgh[vocab_dict["pittsburgh"]] = 1.0
print(e_pittsburgh)
```

```
[0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 0. 1. 0. 0. 0. 0. 0. 0.]
```

Obviously we can recover our document term frequency vector by just adding together the vectors for each word in the document

$$x_{\text{doc}} = \sum_{\text{word} \in \text{doc}} e_{\text{word}}.$$

The issue here, now stated more formally, is that there is no notion of distance within this space. The (squared) distance between two words is either two (if the words are different) or zero (if the words are the same). That is

$$\|e_{\text{pittsburgh}} - e_{\text{boston}}\|_2^2 = \|e_{\text{pittsburgh}} - e_{\text{database}}\|_2^2 = 2.$$

Despite the fact that we think of "pittsburgh" and "boston" as being "closer" to each other than to "database", our term frequency model doesn't capture this at all.

✓ Word embeddings

This discussion brings us to the topic of "word embeddings". Like the one-hot encoding of words, a word embedding is a vector representation of words, in some space $\mathbb{R}^k$. That is, for every word in the encoder, the word embedding represents this word with a k -dimensional vector. There are a few important differences, though:

1. Euclidean distances in the word embedding (attempt to) correspond to some notion of similarity between the words.
2. The dimensionality of the vector, k , does not need to be the same as the vocabulary size (typically it is a much smaller fixed dimension, like $k = 300$).

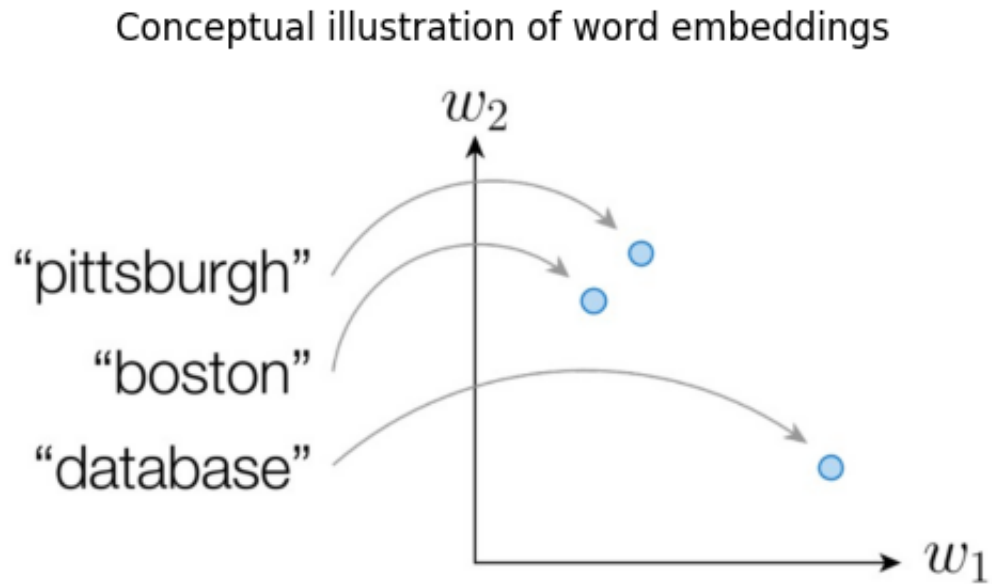
Conceptually, you can think of the embeddings as something like the following figure:

```
from google.colab import drive
drive.mount('/content/drive')
%cd /content/drive/MyDrive/Colab Notebooks/MLandDS/
```

```
Drive already mounted at /content/drive; to attempt to forcibly remount
/content/drive/MyDrive/Colab Notebooks/MLandDS
```

```
import matplotlib.pyplot as plt
from PIL import Image
```

```
image = Image.open("embedding.jpg")
plt.imshow(image)
plt.axis('off')
plt.title('Conceptual illustration of word embeddings')
plt.show()
```



As seen in the figure, "similar" words are mapped to points close together (by Euclidean distance), whereas dissimilar words will be further away.

Word2vec

How do we actually go about creating a word embedding that has these properties, though? We don't yet have enough background to define the algorithms for doing this in detail, but a common approach that has ignited a lot of recent interest in word embeddings is the [word2vec](#) algorithm, see [Mikolov et al., "Efficient Estimation of Word Representations in Vector Space", Arxiv 2013](#) and several variants that arose subsequently.

For our purposes, though, the basic idea of word2vec is that, given a large body of text, we want to train an algorithm that can "predict" the context around a word. For example, suppose we have the following sentence somewhere in our corpus that we are building the word2vec model on:

... eastern US city of Pittsburgh is building new technology centers ...

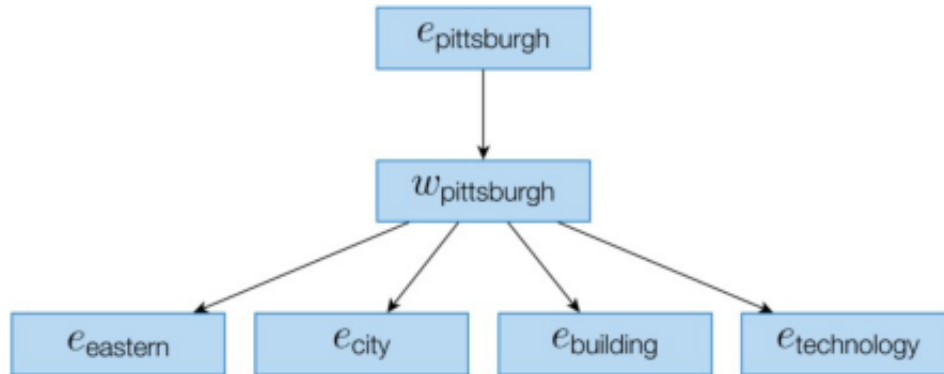
and

.. the eastern city of Boston has well-established technology and biotech sectors but ...

From the word "pittsburgh", we would try to predict some of the surrounding words (possibly ignoring stop words), using a "projection" of pittsburgh into a lower dimensional space. The illustration of the learning task looks like something like the following:

```
image = Image.open("word2vec.jpg")  
plt.imshow(image)  
plt.axis('off')  
plt.title('Illustration of word2vec algorithm')  
plt.show()
```

Illustration of word2vec algorithm



This may seem a bit odd, because there isn't really any way to "predict", with any kind of real accuracy, all the context that will appear around a word just from the word itself. But the idea here is that we try to slightly bump up the probability of the surround words, and lower the probability of other words.

The "reason" why this works is illustrated in the two quotes above. For two words that are very common, there is likely to be substantial overlap in their surrounding context (i.e., the context surrounding the two words is often very similar, whether or not the two actual words occur together). Thus, the two words should make "similar" predictions, which can be achieved by simply making the two word embeddings very similar.

However, in truth there is still a lot of uncertainty about just why the embeddings that these methods generate are so good: although the above logic implies why two word vectors for words with similar context *could* be very close, it doesn't imply that they *will* necessarily be close (and indeed, we could come up with alternative embeddings where both "pittsburgh" and "boston" predicted identical context well, yet had different embeddings. There has been a great deal of follow-on work that has considered the algorithm and variants in much more detail from both an algorithmic and theoretical side, but we won't consider this here.

Pretrained models

While it is absolutely possible to train word2vec models yourself using freely available tools, it is somewhat time consuming, and requires a *very large* corpus to train upon. This is an important point: *building* the word2vec model requires a rich enough data set such that we see similar words in similar contexts. In the example above, you need to see "pittsburgh" and "boston" enough times with enough overlapping context words before the model will start to learn that these two words are similar.

Thus, it is rather unusual to actually build new word2vec models in data science tasks; this would only happen if you have a rich enough data set (and one that deviates enough from the "standard" language that most existing models are trained upon), to build meaningful representations. Trying to build a word2vec model solely on our three-document example above, for instance, would be completely meaningless.

What *is* common, however, is to use existing, pre-trained word2vec models that have already been build based upon large and "standard" corpora. Several people have released such models available for free. For instance, the original word2vec authors released a model trained on over 300 billion words from Google News, and containing 3 million unique words, with embeddings of size $k = 300$, available at the [word2vec project page](#). One problem with this model is that it is relatively large: about 4GB uncompressed in memory, so in practice unless you have more than 16GB of of memory on your system, similarity queries and similar functions will time out. A collections of smaller models is available at the [vsmlib library page](#). And for this particular set of notes we use a version trained on English Wikipedia, originally available [here](#), but which we've compressed a bit and modified to work directly with the gensim library that we use in these notes("deps.words.bin").

▼ Properties of word vectors

Let's look at a few properties of the resulting word vectors. To do so, we're going to use the [gensim](#) library. Gensim was built as a generic topic modeling library (a subject we'll look at a bit more later in the course), but for one purposes now, one of it's nice features is that it has an interface to pre-trained word embedding models. It also has an optimized implementation of the actual word2vec training algorithm, so in the even that you do have a large enough corpus that has some deviations from standard English (a common example in data science would be free-text comments in some specialized topic area, like notes that a technician may write in some very particular application domain), you can also use the library to train your own model.

Loading the pre-trained models, though, is quite straightforward. We can load the model which is mentioned above using the following code.

```
import gensim as gs
import numpy as np
model = gs.models.KeyedVectors.load_word2vec_format('deps.words.bin',
```

We can now get the actual word vectors of many relevant words in the English language (the data set has 174,015 words, so substantially less than the 3 million Google dataset, but still enough to cover a lot of common words you would use in English). We can get the vector of a word using the `.wv` object.

```
model["pittsburgh"][:10]

array([ 0.01674137, -0.04457627, -0.11064089,  0.07378815,
        0.03441498,
        0.05459452,  0.01707146, -0.01835188, -0.10072982,
       -0.08968262],
      dtype=float32)
```

It's important to understand that the values in the word vector itself don't have any absolute meaning, they are only relevant in relation to other word vectors. For instance, we can get the word vectors closest to "pittsburgh" using the `.similar_by_word()` function.

```
model.similar_by_word("pittsburgh", topn=5)
```

```
[('chicago', 0.77215975522995),  
 ('cincinnati', 0.7613445520401001),  
 ('boston', 0.749634325504303),  
 ('detroit', 0.7348772287368774),  
 ('louisville', 0.726100504398346)]
```

This function is actually just a quick wrapper over the more general

`.similar_by_vector()` function, but leaves out the input word itself (which naturally always has full similarity to itself).

```
model.similar_by_vector(model["pittsburgh"], topn=5)
```

```
[('pittsburgh', 1.0),  
 ('chicago', 0.77215975522995),  
 ('cincinnati', 0.7613445520401001),  
 ('boston', 0.749634325504303),  
 ('detroit', 0.7348772287368774)]
```

These results look very reasonable: the most similar words to Pittsburgh are also all cities in the middle and eastern US. You can experiment with other word, and often they seems surprisingly good, especially given the simple process that is used to actually train the model.

Reasoning by analogy

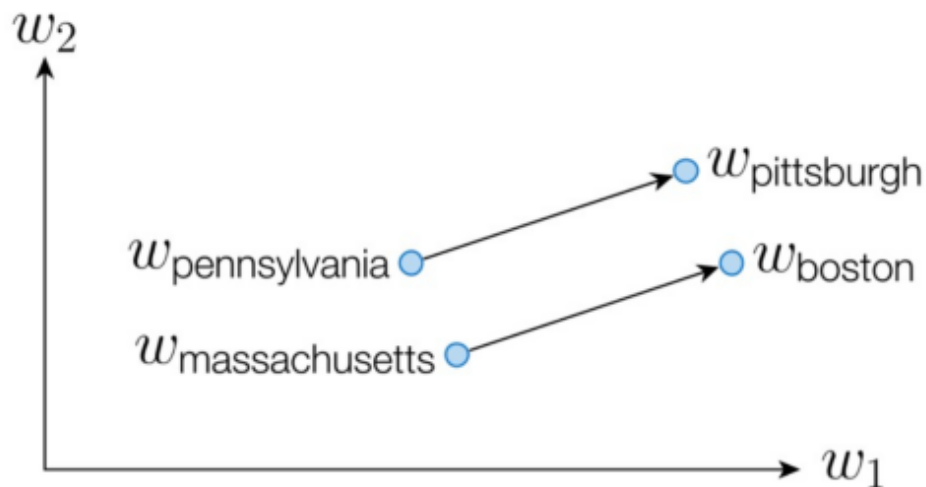
One of the most compelling (and frankly, surprising) properties of word vectors is their ability to perform basic analogical reasoning through arithmetic in the vector space. For example, in the above similarities, "boston" was only the third-most similar word to "pittsburgh". However, what if we consider an analogy like the following:

Pennsylvania : Pittsburgh -> Massachusetts : ?

(read as "Pennsylvania is to Pittsburgh as Massachusetts is to [blank]"). It turns out that in word2vec models, the vectors *between* different vector relationships capture some rudimentary form of this kind of word relationship. Graphically, this looks like the following:

```
image = Image.open("analogy.jpg")
plt.imshow(image)
plt.axis('off')
plt.title('Conceptual illustration of analogy relationships in word embeddings')
plt.show()
```

Conceptual illustration of analogy relationships in word embedding models



We expect the vector from "massachusetts" to "boston" to be similar to the vector of "pennsylvania" to "pittsburgh" (capturing something like, "big cities in the state", or maybe even something more specific like "tech-heavy cities in the state"). To perform this analogical reasoning in vector space, we can get the vector from pennsylvania to pittsburgh ($w_{\text{pittsburgh}} - w_{\text{pennsylvania}}$) and add it to $w_{\text{massachusetts}}$

$$w = w_{\text{pittsburgh}} - w_{\text{pennsylvania}} + w_{\text{massachusetts}}$$

```
model.similar_by_vector(model["pittsburgh"] - model["pennsylvania"] +  
[('pittsburgh', 0.8325108289718628),  
 ('boston', 0.6833232641220093),  
 ('detroit', 0.6712852716445923),  
 ('cincinnati', 0.6654931306838989),  
 ('chicago', 0.6522258520126343)]
```

The most similar word to this vector is just the vector $w_{\text{pittsburgh}}$ (just like the vector for "pittsburgh" was naturally closest to itself, in practice the analogy relationships above are not perfect in the vector space, and since "massachusetts and "pennsylvania" are themselves relatively close, "pittsburgh" still the closest to this modified vector). But after that, "boston" comes in second, relatively more similar to "pittsburgh" than it was before, given the additional information about the states that the two cities are in.

These kind of properties of the word embedding models, are, quite frankly, completely surprising. There is nothing in the training process of word2vec models that optimizes for this property of the resulting vectors. They simply emerge from trying to capture the nearby context of words, given a large enough dataset for training.

▾ Bag of word vectors

Now that we have defined a "nicer" representation of words in a vector space, let's see one simple way of using these to compute document similarity that captures the underlying words. Returning to our previous example of the three documents (two about food in cities, one about PostgreSQL), we can do something similar as to did to create the term frequency document vectors (i.e., add up all the word vectors in the document), but this time using the word2vec embeddings instead of the term frequency one-hot vectors:

$$x_{\text{doc}} = \sum_{\text{word} \in \text{doc}} w_{\text{word}}.$$

where following the notation above, w_{word} denotes the word2vec embedding of the word rather than the one-hot encoding.

We can build these document vectors for our simple corpus with the following code.

```
X_w2v = np.array([sum(model[w.lower()]) for w in doc) for doc in documents])
```

Now let's compute the normalized cosine similarity scores between all the documents.

```
X_w2v = X_w2v / np.linalg.norm(X_w2v, axis=1)[:,None]
M = X_w2v @ X_w2v.T
print(M)
```

```
[[1.          0.7874911  0.69953364]
 [0.7874911  1.          0.76503277]
 [0.69953364 0.76503277 1.          ]]
```

Now our first and second documents (both about food in cities) are the two most similar documents, while each both are less similar to the third, *despite* the fact that the first and second documents have no words in common. Note that in general the similarities here are higher than similarities you get with a TFIDF model, so what is important are the relative similarities, not the absolute similarities. This is because "random" words in a word embedding model still often have some degree of similarity, for example:

```
model.similarity("pittsburgh", "postgresql")
```

```
0.35499242
```

This is certainly low compared to the similarity between "pittsburgh" and "boston", but it's not zero either, and so these terms (and especially the similarity between all the more common words) add up to cause the "absolute" similarities to be relatively high. But again, what is important here is the relative scores, which correctly show that the first two documents are more similar, as we would expect from their content. You can, of course, also combine word2vec embeddings with inverse document frequency weights and similar things, to try to get better representations of the relevant features of documents.

✓ TFIDF in gensim

As a brief note, we can also compute traditional TFIDF scores using gensim (both similarities and the term-frequency matrix in sparse form). We didn't mention this above simply because we hadn't introduced gensim yet, and the TFIDF scores are quite easy to create manually, but this can be handy if you don't want to explicitly create the sparse matrices (though you'll need to create them yourself for the homework).

```
dictionary = gs.corpora.Dictionary(document_words)
corpus = [dictionary.doc2bow(doc) for doc in document_words]
tfidf = gs.models.TfidfModel(corpus)
X_tfidf = gs.matutils.corpus2csc(tfidf[corpus])
print(X_tfidf.todense().T)
```

```
[[0.40824829 0.40824829 0.40824829 0.40824829 0.40824829 0.40824829
  0.          0.          0.          0.          0.          0.
  0.          0.          0.          0.          0.          0.
  0.          0.          0.          0.          0.          0.
  0.16073254 0.43550663 0.43550663 0.43550663 0.43550663 0.16073254
  0.43550663 0.          0.          0.          0.          0.
  0.          0.          0.          0.          0.          0.
  0.16073254 0.          0.          0.          0.          0.16073254
  0.          0.43550663 0.43550663 0.43550663 0.43550663 0.43550663]]
```

Note that the term frequency matrix is actually the transpose of what we defined, but it is of course equivalent. You can also get the cosine similarity matrix as well.

```
M = gs.similarities.MatrixSimilarity(tfidf[corpus])
print(M.get_similarities(tfidf[corpus]))
```

WARNING:gensim.similarities.docsim:scanning corpus to determine the num

```
[[1.      0.      0.      ]
 [0.      1.      0.0516699]
 [0.      0.0516699 1.      ]]
```

3.3. Language models and n-grams

Finally, we provide a very brief introduction to language models, in particular simple n-gram based language models.

Although the bag of words representation does work remarkably well in many cases, it of course is still an incredibly simple representation of documents that will miss basic properties. For example the documents

great moving and not boring

has the exact same bag of words representation (for both term frequency and word2vec models) as the document

boring movie and not great

because they contain the exact same words. As despite the fact, mentioned above, that we don't often talk like this, we would like to consider a better way to represent more complex structure in language.

Probabilistic language models

A probabilistic language model attempts to predict the probability of each word in the document, given all the words that proceeded it

$$P(\text{word}_i | \text{word}_1, \dots, \text{word}_{i-1})$$

(we are going to cover probability more later in this course, but the notions of probability here are very simple, just counting, and so with some apologies for the forward reference, we're going to dive into the subject now as it fits best the free text section). This is natural representation for language, because we do intuitively feel like the proceeding words in a sentence give some distribution about what the subsequent words could be. For example, consider the partial sentence

Data science is the study and practice of how we can extract insight and knowledge from large amounts of

While we may not be able to prescribe precise numerical probabilities to the next word (humans are quite bad as prescribing probabilities anyway), there is an obvious sense in which the probability

$$P(\text{word}_i = \text{"data"} | \text{word}_1, \dots, \text{word}_{i-1})$$

is greater than the probability

$$P(\text{word}_i = \text{"hotdogs"} | \text{word}_1, \dots, \text{word}_{i-1}).$$

Of course, note that this also requires some information about language (which we maybe could extract from a word2vec model, though we won't get even that complex here), to understand that "data" makes more sense given that the sentence is talking about data science.

✓ N-gram models

Building a language model that uses the entirety of preceeding text in a generic manner, capturing long term dependencies and bringing in "common sense" knowledge, is a very much still a long-term goal of natural language processing, and we won't be considering what is the current state of the art for such approaches. Instead, our goal is to build a rudimentary language model that captures some basic context of the preceeding words in predicting the next one.

The technique we will use to do this, which is still an absolute standard baseline for many NLP tasks (if you can't beat it with a fancy model, then it is a strong sign that your model may not be so useful after all), is the *n-gram model*. The idea here is to put a *cap* on the size of the context we use to predict the next word, so that under an n-gram model we have that

$$P(\text{word}_i | \text{word}_1, \dots, \text{word}_{i-1}) = P(\text{word}_i | \text{word}_{i-n+1}, \dots, \text{word}_{i-1})$$

i.e., we only use the $n - 1$ previous terms to predict the next term. So in a 4-gram model, we would predict the probabilities

$P(\text{word}_i = \text{"data"} | \text{word}_1, \dots, \text{word}_{i-1})$, and $P(\text{word}_i = \text{"hotdogs"} | \text{word}_1, \dots, \text{word}_{i-1})$, using just the three previous words, "large amounts of [blank]". Obviously, a lot of context is being thrown away here, and you would imagine that the probability of "hotdogs" is going to be much higher in this context than the probability of "hotdogs" being the next word in the original sentence (even if it still likely has lower probability than "data", simply because people talk about data more than they text to talk about hotdogs).

N-gram models are typically *built* based upon some fixed corpus, and when we refer to probabilities in an n-gram model, we are just referring to counts in this corpus. That is given some corpus,

$$P(\text{word}_i | \text{word}_{i-n+1}, \dots, \text{word}_{i-1}) = \frac{\#(\text{word}_{i-n+1}, \dots, \text{word}_i)}{\#(\text{word}_{i-n+1}, \dots, \text{word}_{i-1})}$$

i.e., we just count the number of times we have seen the entire sequence of words $\text{word}_{i-n+1}, \dots, \text{word}_i$ divided by the number of times we have seen just the preceding words $\text{word}_{i-n+1}, \dots, \text{word}_{i-1}$. For example,

$$P(\text{"data"} | \text{"large amounts of"}) = \frac{\#(\text{"large amounts of data"})}{\#(\text{"large amounts of"})}$$

is just the number of times we see the entire phrase "large amounts of data" divided by the times we see the phrase "large amounts of", in our training corpus. This is as complex as the probabilities are going to get in our setting.

Laplace smoothing One potential issue with this basic n -gram model is that it will frequently predict that the probability of some phrase is zero, if that phrase has never been seen before in the corpus we used to build the model. This can be problematic, especially if we want to evaluate the probability of text outside the corpus we used to build the model (in which case, there are almost definitely going to be phrases in the new text that we hadn't seen before). One simple strategy that alleviates this problem somewhat (though it does so in a very naive way), is to add what is called *Laplace smoothing*, which just adds a small probability to every possible next word

$$P(\text{word}_i | \text{word}_{i-n+1}, \dots, \text{word}_{i-1}) = \frac{\#(\text{word}_{i-n+1}, \dots, \text{word}_i) + \alpha}{\#(\text{word}_{i-n+1}, \dots, \text{word}_{i-1}) + \alpha D}$$

where D is the number of total unique words in the vocabulary and α is some small number. A (slightly) more sophisticated approach might back off to lower n in the n -gram model when it encounters a phrase it has not seen before; or a (much more complex) model might actually estimate these probabilities to some extent using similarities between words such as those based upon word embeddings.

Choosing n How do we choose the best value for n ? If we choose n to be larger, we have more context in modeling our probabilities, but we also need more data until we are able to get a good estimate of the relevant probabilities. On the other hand, smaller n will use less context, but also require less data to model the probabilities.

While the correct answer about choosing the right n is to use some evaluation metric on a held out data set (we'll discuss this kind of methodology a lot more when we discuss machine learning in general), in practice it's common to simply choose $n = 2$ when we have a "small" training corpus, up to hundreds of thousands of words (i.e., predict based upon just the previous word), and $n = 3$ when we have a larger training corpus, more than a million words (predict based upon two previous words). These are, of course, just very rough rules of thumb, and you'll look into this a bit more in your homework.

Using language models in data science

How might these models be used in a data science context? One nice application of language models is in author identification or de-anonymization. If we have two large corpora of work C_1 and C_2 from two different authors, then we can build an n -gram language model based upon the first corpora, and another based upon the second corpora. Then, if we have a new document that we believe to be written by one of the two authors, but we do not know which one, we can then use these two different models to compute the probability of the new document under each of these two models. We can then guess that the true author of the document is the person whose language model gives higher probability (evaluated e.g., via perplexity) to the unknown document.

Sampling from the models

A perhaps less-practical (but amusing) application of language models is to generate synthetic text that is "similar" in some way to the text used to build the model. Naturally, a simple n -gram model will not be able to generate text with any real semantic content (it is arguable whether any model that we currently have is capable of this), but the generated text is often amusingly similar to the source material.

We built n -gram models, with varying n , on the complete works of Shakespeare, available in a single file [here](#). We don't include the full source here, but the basic model is quite simple, just dictionaries/counters of all $n - 1$ and n word combinations present in the text, used to compute the probabilities as above (some care needs to be taken in creating an efficient structure to look up all occurring next-words given the $n - 1$ previous words).

Then, we *randomly sample* from this model. That is, given the previous $n - 1$ words, we compute the probability of each next word from the entire vocabulary, and we randomly sample from this distribution (when we have seen no words so far, we first sample from the 1-gram distribution, then 2-gram, etc, up to n -gram). We repeat this process some fixed number of times to generate the synthetic text.

Here is some generated by the $n = 1$ model (in our model, punctuation was considered to be its own word):

in as , stands gods revenge ! france pitch good in fair hoist an what fair
shallow-rooted , . that with wherefore it what a as your . , powers course
which thee dalliance all

This looks like some words that Shakespeare might use, but in an $n = 1$ model, remember that we are using *no* previous context to predict the next word. That is, we are simply randomly sampling single words one after another from the set and frequency of all words used by Shakespeare; so it is not too surprising that the text doesn't look like anything meaningful. Now let's consider the $n = 2$ model.

look you may i have given them to the dank here to the jaws of tune of
great difference of ladies . o that did condemn what of ear is shorter
time ; yet seems to

This seems a bit better. There is no repeated punctuation, and two-word pairs here all seem reasonable, but it still doesn't look much like real written text, at all. Finally, $n = 3$:

believe , they all confess that you withhold his levied host , having
brought the fatal bowels of the pope ! ' and that this distemper'd
messenger of heaven , since thou deniest the gentle desdemona

This is now becoming fairly realistic. There is, of course, no real content to the sentences, but they are a good source of material that looks superficially like the original source (several people have, for example, build n-gram language models for various public figures with distinctive writing styles).

Let's go a bit further and consider text from an $n = 7$ model:

so express'd : but what of that ? 'twere good you do so much for charity
. i cannot find it ; 'tis not in the bond . you , merchant , have you any
thing to say ? but little

This is starting to look a lot like Shakespeare, because it *is* Shakespeare. Specifically, it is a passage from Act IV, Scene I of The Merchant of Venice. What happens in this case is that because our context is so long, looking at the past 6 words, once we have generated some text there is only *one* instance in all of Shakespeare of these particular set of six words occurring; thus, we will, with probability 1.0, just "sample"

the word that occurs immediately after, and repeat the process. Note that this only occurs when we do not use Laplace smoothing, but in the case that we do use Laplace smoothing, something just as bad happens: we typically get "off" from any text that ever has occurred in Shakespeare, so both the numerator and denominator counts are zero, and we just randomly sample words.

Evaluating language models

Other than simply generating some random text and seeing what "looks right" (and as we saw, this can be deceiving if we start to generate a lot of text that is identical to the source), how might we evaluate how well a given model is really fitting to data. The most common metric used in NLP to see how well a language model captures some text is the *perplexity* metric.

Perplexity is defined as a (scaled) version of the inverse probability of the data, specifically, given a document with N words

$$\text{Perplexity} = \left(\frac{1}{P(\text{word}_1, \dots, \text{word}_N)} \right)^{\frac{1}{N}} = \exp \left(\frac{-\log P(\text{word}_1, \dots, \text{word}_N)}{N} \right)$$

Note that we can compute the probability $P(\text{word}_1, \dots, \text{word}_N)$ in an n -gram model by

$$P(\text{word}_1, \dots, \text{word}_N) = \prod_{i=n}^N P(\text{word}_i | \text{word}_{i-n+1}, \dots, \text{word}_{i-1})$$

where for simplicity we just ignore the first $n - 1$ words so we don't need to worry about modeling probabilities with a smaller context (if the document is large enough, this will have very small effect).

One thing to note here is that this formula involves multiplying together very small probabilities. In general, you don't want to do this, but instead you want to compute the *log* of this quantity, given by

$$\log P(\text{word}_1, \dots, \text{word}_N) = \sum_{i=n}^N \log P(\text{word}_i | \text{word}_{i-n+1}, \dots, \text{word}_{i-1})$$

and then use this quantity directly in the second equivalent version of perplexity above. Note, however, that if the probability of any observed sequence of n terms is *exactly* zero, then the entire probability of the document will also be zero. Thus, when evaluating n -gram language models on anything other than the training sequence, you will typically always want to employ Laplace smoothing.

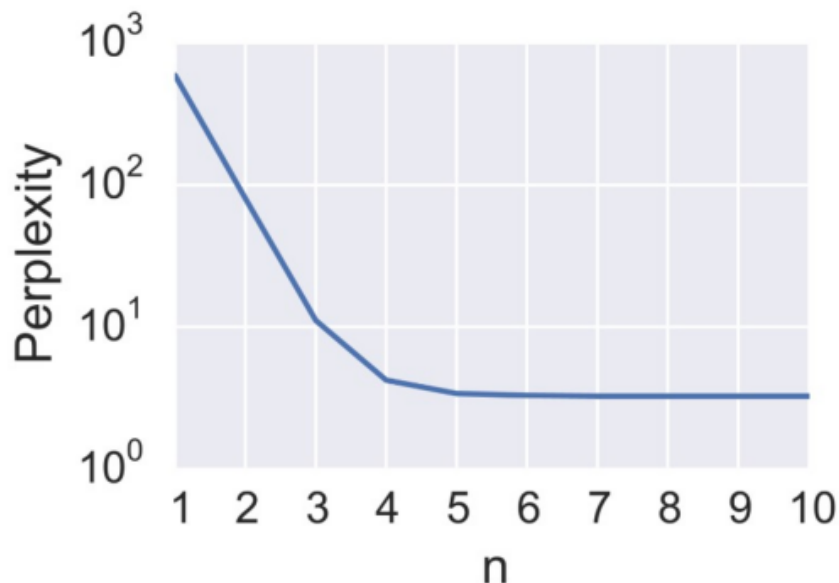
Training and validation sets

Notice that the perplexity of the language model evaluated on the training set (the corpus used to build the model), will always decrease with greater n ; as seen in the example above, larger n just gives higher probability to the exact sequences we saw in the training corpus. The following is a plot of the perplexity of our Shakespeare model evaluated on the data it was built on (note that we use Laplace smoothing here, so that the perplexity does not decrease to 1, as it would if we didn't use smoothing).

```

image = Image.open("perpl_training.jpg")
plt.imshow(image)
plt.axis('off')
plt.title('Perplexity versus the size of the n-gram model, for text u:
plt.show()
  
```

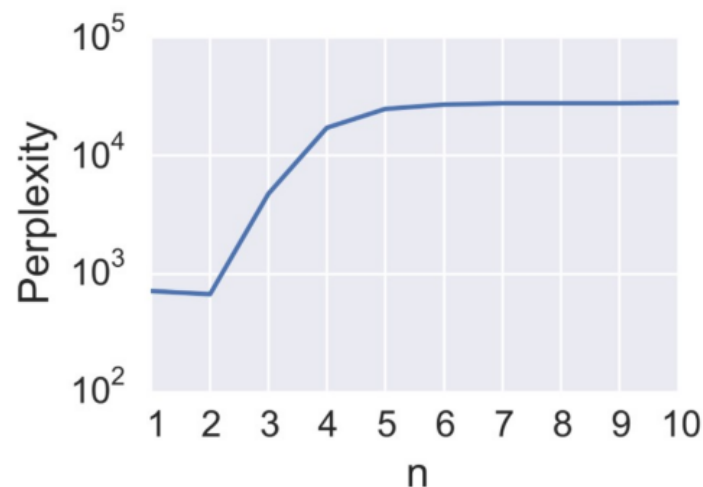
Perplexity versus the size of the n-gram model, for text used to build the n-gram model



On the other hand, if we evaluate the model on text *other* than what we used to build it (in this case, we build the model only on the first 70% of the Shakespeare works, and use this model to compute probabilities on the remaining 30%, a set called *hold-out validation set*), the situations looks very different. Our perplexity decreases initially, but quickly increases to very large amounts, simply because none of the past phrases are even seen in the text, and so we rely solely on Laplace smoothing to assign any probability to the words at all.

```
image = Image.open("perpl_testing.jpg")
plt.imshow(image)
plt.axis('off')
plt.title('Perplexity versus the size of the n-gram model, for a new (
plt.show()
```

Perplexity versus the size of the n-gram model, for a new document that the model was not trained on



From a perplexity standpoint, it actually seems like $n = 2$ is the best setting for Shakespeare, as even the $n = 3$ model fits to strongly to what was seen in the training corpus.

✓ 4. Advanced NLP Techniques

✓ 4.1. Recurrent Neural Networks (RNNs)

Overview of RNNs in NLP:

- Recurrent Neural Networks (RNNs) are a type of neural network designed to handle sequential data by maintaining a 'memory' of previous inputs, which is particularly useful in tasks involving text, speech, or time series data.
- In NLP, RNNs are used for tasks like language modeling, text generation, machine translation, and sentiment analysis.
- RNNs suffer from issues like vanishing and exploding gradients, which makes training deep RNNs difficult for long sequences.

Introduction to LSTM and GRU Networks:

- **Long Short-Term Memory (LSTM):**
 - LSTM networks are a type of RNN that can learn long-term dependencies. They use a gating mechanism (input, forget, and output gates) to control the flow of information and mitigate the vanishing gradient problem.
- **Gated Recurrent Units (GRU):**
 - GRUs are a simplified version of LSTMs that use only two gates (reset and update gates), making them computationally more efficient while still effectively capturing dependencies in sequences.

Example of LSTM for Text Classification:

```
!pip install tensorflow
```

```
Requirement already satisfied: tensorflow in /usr/local/lib/python3.10/
Requirement already satisfied: absl-py>=1.0.0 in /usr/local/lib/python.
Requirement already satisfied: astunparse>=1.6.0 in /usr/local/lib/pytl
Requirement already satisfied: flatbuffers>=24.3.25 in /usr/local/lib/p
Requirement already satisfied: gast!=0.5.0,!0.5.1,!0.5.2,>=0.2.1 in /
Requirement already satisfied: google-pasta>=0.1.1 in /usr/local/lib/py
Requirement already satisfied: h5py>=3.10.0 in /usr/local/lib/python3.
Requirement already satisfied: libclang>=13.0.0 in /usr/local/lib/pytho
Requirement already satisfied: ml-dtypes<0.5.0,>=0.3.1 in /usr/local/l
Requirement already satisfied: opt-einsum>=2.3.2 in /usr/local/lib/pytl
Requirement already satisfied: packaging in /usr/local/lib/python3.10/c
Requirement already satisfied: protobuf!=4.21.0,!4.21.1,!4.21.2,!4.
Requirement already satisfied: requests<3,>=2.21.0 in /usr/local/lib/py
Requirement already satisfied: setuptools in /usr/local/lib/python3.10/
Requirement already satisfied: six>=1.12.0 in /usr/local/lib/python3.10
Requirement already satisfied: termcolor>=1.1.0 in /usr/local/lib/pytho
Requirement already satisfied: typing-extensions>=3.6.6 in /usr/local/
Requirement already satisfied: wrapt>=1.11.0 in /usr/local/lib/python3
Requirement already satisfied: grpcio<2.0,>=1.24.3 in /usr/local/lib/py
Requirement already satisfied: tensorboard<2.18,>=2.17 in /usr/local/l
Requirement already satisfied: keras>=3.2.0 in /usr/local/lib/python3.
Requirement already satisfied: tensorflow-io-gcs-filesystem>=0.23.1 in
Requirement already satisfied: numpy<2.0.0,>=1.23.5 in /usr/local/lib/p
Requirement already satisfied: wheel<1.0,>=0.23.0 in /usr/local/lib/py
Requirement already satisfied: rich in /usr/local/lib/python3.10/dist-
Requirement already satisfied: namex in /usr/local/lib/python3.10/dist-
Requirement already satisfied: optree in /usr/local/lib/python3.10/dis
Requirement already satisfied: charset-normalizer<4,>=2 in /usr/local/
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/py
Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/py
Requirement already satisfied: markdown>=2.6.8 in /usr/local/lib/python
Requirement already satisfied: tensorboard-data-server<0.8.0,>=0.7.0 in
Requirement already satisfied: werkzeug>=1.0.1 in /usr/local/lib/python
Requirement already satisfied: MarkupSafe>=2.1.1 in /usr/local/lib/pytl
Requirement already satisfied: markdown-it-py>=2.2.0 in /usr/local/lib/
Requirement already satisfied: pygments<3.0.0,>=2.13.0 in /usr/local/l
Requirement already satisfied: mdurl~=0.1 in /usr/local/lib/python3.10/
```

```
import numpy as np
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Embedding, LSTM, Dense, Bidirectio
from tensorflow.keras.preprocessing.text import Tokenizer
from tensorflow.keras.preprocessing.sequence import pad_sequences
```

```
# Sample data
texts = ['I love machine learning', 'Deep learning is amazing', 'NLP :
labels = [1, 1, 1] # Assuming binary classification for simplicity

# Tokenize and pad sequences
tokenizer = Tokenizer(num_words=10000)
tokenizer.fit_on_texts(texts)
sequences = tokenizer.texts_to_sequences(texts)
X = pad_sequences(sequences, maxlen=10)

# Convert labels to numpy array
y = np.array(labels)

# Define the LSTM model with clear input shape
model = Sequential([
    Embedding(input_dim=10000, output_dim=64), # input_length is auto
    Bidirectional(LSTM(64, return_sequences=False)),
    Dense(1, activation='sigmoid')
])

# Compile and display model summary correctly
model.build(input_shape=(None, 10)) # Explicitly build the model with
model.compile(optimizer='adam', loss='binary_crossentropy', metrics=[
model.summary()

# Example data
X_train, y_train = X, y

# Train the model
model.fit(X_train, y_train, epochs=5, batch_size=2)
```

Model: "sequential_1"

Layer (type)	Output Shape
embedding_1 (Embedding)	(None, 10, 64)
bidirectional_1 (Bidirectional)	(None, 128)
dense_1 (Dense)	(None, 1)

Total params: 706,177 (2.69 MB)

Trainable params: 706,177 (2.69 MB)

Non-trainable params: 0 (0.00 B)

Epoch 1/5

2/2 ————— 6s 23ms/step - accuracy: 0.6111 - loss: 0.6882

```
Epoch 2/5
2/2 ----- 0s 22ms/step - accuracy: 1.0000 - loss: 0.6448
Epoch 3/5
2/2 ----- 0s 21ms/step - accuracy: 1.0000 - loss: 0.5982
Epoch 4/5
2/2 ----- 0s 28ms/step - accuracy: 1.0000 - loss: 0.5451
Epoch 5/5
2/2 ----- 0s 21ms/step - accuracy: 1.0000 - loss: 0.4804
```

✓ 4.2. Transformer Models

Introduction to Transformer Architecture:

- Transformers are a type of neural network architecture that relies entirely on self-attention mechanisms, dispensing with recurrence and convolution entirely.
- They are highly effective for processing sequential data because they allow parallelization, which significantly speeds up training and improves model performance.
- Transformers use multi-head self-attention and positional encoding to handle sequential data without relying on recurrence.

Overview of BERT, GPT, and Other Modern Models:

- **BERT (Bidirectional Encoder Representations from Transformers):**
 - BERT is designed to understand the context of a word based on all its surrounding words in a sentence (bidirectional).
 - It is pre-trained on a large corpus of text and fine-tuned for specific tasks.
- **GPT (Generative Pre-trained Transformer):**
 - GPT is designed for generating coherent text by predicting the next word in a sequence (unidirectional).
 - GPT-3, one of the largest models, is widely used for text generation tasks.
- **Other Models:**
 - **RoBERTa:** An optimized version of BERT with improved training procedures.
 - **T5 (Text-To-Text Transfer Transformer):** A versatile model that treats every NLP task as a text-to-text problem.

Example Using BERT for Text Classification:

```

from transformers import BertTokenizer, TFBertForSequenceClassification
from tensorflow.keras.optimizers import Adam
import tensorflow as tf

# Load the tokenizer and model
tokenizer = BertTokenizer.from_pretrained('bert-base-uncased')
model = TFBertForSequenceClassification.from_pretrained('bert-base-uncased')

# Example text
texts = ["I love NLP!", "Transformers are powerful."]

# Tokenize and prepare the input
inputs = tokenizer(texts, padding=True, truncation=True, return_tensors='tf')

# Dummy labels
labels = tf.convert_to_tensor([1, 0])

# Compile the model using a loss that is compatible with BERT outputs
optimizer = Adam(learning_rate=5e-5)

# Convert inputs for model
def compute_loss(labels, logits):
    return tf.nn.sparse_softmax_cross_entropy_with_logits(labels=labels, logits=logits)

# Custom training loop
for epoch in range(2): # loop over the dataset multiple times
    with tf.GradientTape() as tape:
        outputs = model(inputs['input_ids'], labels=labels)
        loss = outputs.loss
    gradients = tape.gradient(loss, model.trainable_variables)
    optimizer.apply_gradients(zip(gradients, model.trainable_variables))
    print(f'Epoch {epoch+1}, Loss: {loss.numpy()}')

```

All PyTorch model weights were used when initializing TFBertForSequenceClassification.

Some weights or buffers of the TF 2.0 model TFBertForSequenceClassification were not initialized.
 You should probably TRAIN this model on a down-stream task to be able to use it for predictions.

Epoch 1, Loss: [0.8075237 0.57690924]
 Epoch 2, Loss: [0.6079752 0.42471638]

✓ Neural Network Transformers (Examples)

```
# Install the transformers and torch libraries
!pip install transformers torch
```

Semantic Analysis

```
import torch
from transformers import BertTokenizer, BertForSequenceClassification
from transformers import TextClassificationPipeline

# Load pre-trained BERT model and tokenizer
model_name = "nlpTown/bert-base-multilingual-uncased-sentiment"
tokenizer = BertTokenizer.from_pretrained(model_name)
model = BertForSequenceClassification.from_pretrained(model_name)

# Create a text classification pipeline
classifier = TextClassificationPipeline(model=model, tokenizer=tokenizer)

# Example usage: classify a list of sentences
texts = [
    "I love this product! It's amazing.",
    "This is the worst experience I've ever had.",
    "Not bad, but could be better.",
    "Absolutely fantastic! Highly recommend.",
    "Terrible service, would not come back."
]

# Classify each text
for text in texts:
    result = classifier(text)
    print(f"Text: {text}")
    print(f"Sentiment: {result[0]['label']} with score {result[0]['score']}
```

```
/usr/local/lib/python3.10/dist-packages/huggingface_hub/utils/_token.py:31:
The secret `HF_TOKEN` does not exist in your Colab secrets.
To authenticate with the Hugging Face Hub, create a token in your settings
You will be able to reuse this secret in all of your notebooks.
Please note that authentication is recommended but still optional to access
warnings.warn(
tokenizer_config.json: 0%|          | 0.00/39.0 [00:00<?, ?B/s]
vocab.txt: 0%|          | 0.00/872k [00:00<?, ?B/s]
special_tokens_map.json: 0%|          | 0.00/112 [00:00<?, ?B/s]
/usr/local/lib/python3.10/dist-packages/huggingface_hub/file_download.py:110:
warnings.warn(
```

```
config.json: 0%|          | 0.00/953 [00:00<?, ?B/s]
pytorch_model.bin: 0%|          | 0.00/669M [00:00<?, ?B/s]
Text: I love this product! It's amazing.
Sentiment: 5 stars with score 0.9424
```

```
Text: This is the worst experience I've ever had.
Sentiment: 1 star with score 0.9380
```

```
Text: Not bad, but could be better.
Sentiment: 3 stars with score 0.7308
```

```
Text: Absolutely fantastic! Highly recommend.
Sentiment: 5 stars with score 0.9823
```

```
Text: Terrible service, would not come back.
Sentiment: 1 star with score 0.8833
```

Next Sentence Prediction

```

import torch
from transformers import BertTokenizer, BertForNextSentencePrediction

# Load pre-trained BERT model and tokenizer
model_name = 'bert-base-uncased'
tokenizer = BertTokenizer.from_pretrained(model_name)
model = BertForNextSentencePrediction.from_pretrained(model_name)

# Function to predict if the second sentence is the next sentence
def is_next_sentence(sentence_a, sentence_b):
    encoding = tokenizer.encode_plus(sentence_a, sentence_b, return_tensors='pt')
    outputs = model(**encoding, labels=torch.LongTensor([1]))
    logits = outputs.logits
    probs = torch.softmax(logits, dim=-1)
    return probs[0][0].item(), probs[0][1].item()

# Example usage
sentence_a = "The sky is black."
sentence_b = "The stars are shining."

prob_not_next, prob_is_next = is_next_sentence(sentence_a, sentence_b)
print(f"Probability that '{sentence_b}' is the next sentence after '{sentence_a}': {prob_is_next}")
print(f"Probability that '{sentence_b}' is NOT the next sentence after '{sentence_a}': {prob_not_next}")

```

```

Probability that 'The stars are shining.' is the next sentence after 'The sky is black.': 0.0001
Probability that 'The stars are shining.' is NOT the next sentence after 'The sky is black.': 0.9999

```

Machine Translation

```
!pip install sacremoses
```

Collecting sacremoses

Downloading sacremoses-0.1.1-py3-none-any.whl (897 kB)

```

897.5/897.5 kB 11.3 MB/s
Requirement already satisfied: regex in /usr/local/lib/python3.10/dist-packages (2022.10.31)
Requirement already satisfied: click in /usr/local/lib/python3.10/dist-packages (8.1.3)
Requirement already satisfied: joblib in /usr/local/lib/python3.10/dist-packages (1.2.0)
Requirement already satisfied: tqdm in /usr/local/lib/python3.10/dist-packages (4.64.1)
Installing collected packages: sacremoses
Successfully installed sacremoses-0.1.1

```

```
from transformers import MarianMTModel, MarianTokenizer

# Load pre-trained MarianMT model and tokenizer for English to French
model_name = 'Helsinki-NLP/opus-mt-en-fr'
tokenizer = MarianTokenizer.from_pretrained(model_name)
model = MarianMTModel.from_pretrained(model_name)

# Function to translate text from English to French
def translate(text, model, tokenizer):
    inputs = tokenizer.encode(text, return_tensors='pt')
    translated = model.generate(inputs, max_length=40, num_beams=4, eos_token_id=tokenizer.eos_token_id)
    translated_text = tokenizer.decode(translated[0], skip_special_tokens=True)
    return translated_text

# Example usage
text = "The sun is shining."
translated_text = translate(text, model, tokenizer)
print(f"Translated text: {translated_text}")
```

```
tokenizer_config.json: 0%|          | 0.00/42.0 [00:00<?, ?B/s]
source.spm: 0%|          | 0.00/778k [00:00<?, ?B/s]
target.spm: 0%|          | 0.00/802k [00:00<?, ?B/s]
vocab.json: 0%|          | 0.00/1.34M [00:00<?, ?B/s]
config.json: 0%|          | 0.00/1.42k [00:00<?, ?B/s]
pytorch_model.bin: 0%|          | 0.00/301M [00:00<?, ?B/s]
generation_config.json: 0%|          | 0.00/293 [00:00<?, ?B/s]
Translated text: Le soleil brille.
```

✓ 4.3 Transfer Learning in NLP

Fine-Tuning Pre-Trained Models for Specific Tasks:

- Transfer learning in NLP involves taking a pre-trained model (e.g., BERT, GPT) and fine-tuning it on a smaller, task-specific dataset.
- This approach leverages the extensive training that the pre-trained model has already undergone on a large corpus, allowing the model to adapt to new tasks quickly with minimal additional training.
- Fine-tuning typically involves adjusting the weights of the pre-trained model on a new dataset while using a lower learning rate to avoid overfitting.

Summary:

- **RNNs (LSTM and GRU):** Useful for capturing sequential dependencies in text data but have limitations in handling long-term dependencies.
- **Transformer Models:** State-of-the-art models in NLP that leverage self-attention mechanisms, enabling parallel processing and better handling of sequential data without recurrence.
- **Transfer Learning:** Fine-tuning pre-trained models like BERT and GPT allows leveraging extensive pre-training for specific tasks with minimal additional training, improving performance and reducing the need for large task-specific datasets.

These advanced techniques have revolutionized NLP, making it possible to achieve high accuracy on complex tasks with relatively less task-specific data.

✓ 5. NLP Project Pipeline

The NLP (Natural Language Processing) project pipeline encompasses the end-to-end process of developing and deploying NLP models. It includes steps from data collection to model evaluation, ensuring the model is robust, reliable, and performs well on real-world data. Below is a detailed explanation of each part of the pipeline:

Data Collection:

Sources for NLP Datasets:

- **Kaggle:** Offers a wide variety of NLP datasets, such as sentiment analysis datasets, language translation datasets, and more. Examples include the IMDb movie reviews dataset and the Twitter US Airline Sentiment dataset.
- **UCI Machine Learning Repository:** Provides numerous text datasets suitable for NLP tasks, such as spam detection, sentiment analysis, and topic classification.
- **Public APIs:** APIs like Twitter API, Reddit API, and News API can be used to collect live data. These sources are useful for building datasets that reflect current events or trends.

Ethical Considerations in Text Data Collection:

- **Consent and Privacy:** Ensure data is collected in compliance with privacy laws and regulations, such as GDPR. Collect data only if you have the right to do so and ensure anonymity where required.
- **Bias and Fairness:** Be aware of biases in the collected data, which can affect model performance and fairness. Datasets should represent the target population adequately.
- **Copyright and Licensing:** Make sure to respect copyright laws and use data that is publicly available or licensed for use.

✓ Exploratory Data Analysis (EDA) for Text:

Visualizing Text Data Distributions:

- **Word Cloud:** A graphical representation of word frequency, where more frequent words appear larger. Useful for understanding the most common terms in your dataset.
- **Bar Plots and Histograms:** Can be used to visualize the frequency of different classes or the distribution of document lengths in the dataset.

Common EDA Techniques for Text Data:

- **Token Frequency Analysis:** Identifies the most common tokens (words) and their frequencies, helping to understand the dataset's vocabulary.
- **N-gram Analysis:** Examines sequences of words (bigrams, trigrams) to uncover common phrases or terms that may be important for the task.
- **Sentiment Distribution:** Visualizes the sentiment distribution in the dataset, useful for tasks like sentiment analysis.

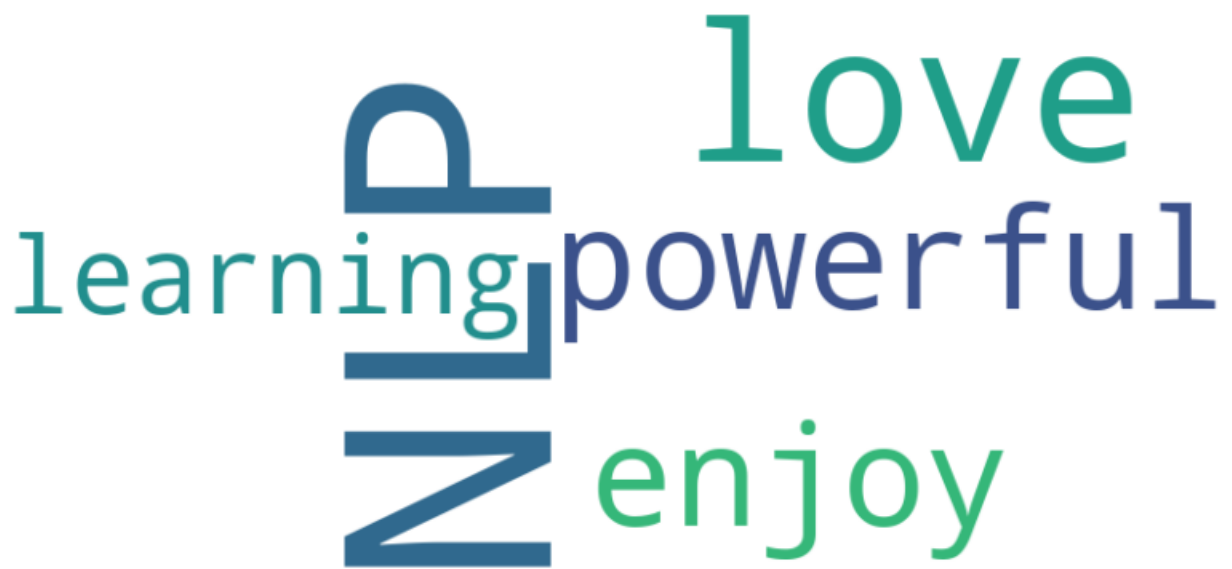
Example for EDA:

```
import matplotlib.pyplot as plt
from wordcloud import WordCloud
import pandas as pd
from collections import Counter

# Example text data
texts = ["I love NLP!", "NLP is powerful.", "I enjoy learning about NLP"]

# Word cloud
all_words = ' '.join(texts)
wordcloud = WordCloud(width=800, height=400, background_color='white')
plt.figure(figsize=(10, 5))
plt.imshow(wordcloud, interpolation='bilinear')
plt.axis('off')
plt.show()

# Token frequency
tokens = all_words.split()
token_counts = Counter(tokens)
print("Token Frequency:", token_counts)
```



```
Token Frequency: Counter({'I': 2, 'love': 1, 'NLP!': 1, 'NLP': 1, 'is': 1, 'learning': 1, 'powerful': 1, 'enjoy': 1})
```

✓ **Model Building:**

Selecting the Right Model for the Task:

- **Classification:** For tasks like sentiment analysis or spam detection, models such as Logistic Regression, Naive Bayes, and Transformer models (e.g., BERT) are commonly used.
- **Generation:** For text generation tasks, models like GPT (Generative Pre-trained Transformer) are suitable.
- **Sequence Labeling:** For tasks like named entity recognition (NER), models like CRF (Conditional Random Fields) or BERT-based models are often used.

Training and Evaluating NLP Models:

- **Training:** Involves feeding labeled data into the model and adjusting parameters to minimize error. Modern NLP models like BERT or GPT are typically fine-tuned on specific tasks.
- **Evaluation:** After training, models are evaluated using a separate test dataset to measure performance on unseen data.

Example for Model Training:

```

from sklearn.model_selection import train_test_split
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.naive_bayes import MultinomialNB
from sklearn.metrics import accuracy_score

# Sample dataset
texts = ["I love NLP!", "NLP is powerful.", "I enjoy learning about NLP",
        "NLP is challenging.", "I dislike bad NLP models."]
labels = [1, 1, 1, 0, 0]

# Vectorize text data
vectorizer = CountVectorizer()
X = vectorizer.fit_transform(texts)
y = labels

# Use stratified split to maintain the distribution of classes in both
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.4, random_state=42, stratify=y
)

# Train a simple Naive Bayes model
model = MultinomialNB()
model.fit(X_train, y_train)

# Evaluate the model
y_pred = model.predict(X_test)
print(f"Accuracy: {accuracy_score(y_test, y_pred)}")

```

Accuracy: 0.5

✓ Model Evaluation Metrics:

Common Evaluation Metrics:

- **Accuracy:** Measures the proportion of correct predictions among the total number of cases examined.
- **Precision:** The ratio of correctly predicted positive observations to the total predicted positives. Indicates the accuracy of positive predictions.
- **Recall:** The ratio of correctly predicted positive observations to all observations in actual class. It tells how many of the actual positives were captured.
- **F1-Score:** The weighted average of Precision and Recall, useful for imbalanced datasets.
- **BLEU (Bilingual Evaluation Understudy):** Used for evaluating text generation tasks by comparing generated text to reference texts.
- **ROUGE (Recall-Oriented Understudy for Gisting Evaluation):** Used for summarization tasks, comparing overlap between generated and reference summaries.

Example of Evaluation Code:

```
from sklearn.metrics import precision_score, recall_score, f1_score

# Assume y_test and y_pred are available from the previous code
precision = precision_score(y_test, y_pred)
recall = recall_score(y_test, y_pred)
f1 = f1_score(y_test, y_pred)
```

```
print(f"Precision: {precision}, Recall: {recall}, F1-Score: {f1}")
```

```
Precision: 0.5, Recall: 1.0, F1-Score: 0.6666666666666666
```

✓ 6. Future Trends in NLP

As Natural Language Processing (NLP) continues to evolve, new technologies and methodologies are emerging that promise to further advance the field. These trends are driven by the need for more efficient, scalable, and versatile NLP systems that can operate across multiple languages, handle low-resource scenarios, and integrate seamlessly with other AI domains. Here, we explore some of the key future trends in NLP:

Emerging Technologies:

1. Low-Resource NLP:

- **Definition:** Low-resource NLP focuses on developing models that can work effectively with limited training data, which is a common scenario for many languages and dialects that lack extensive datasets.
- **Approaches:** Techniques such as transfer learning, data augmentation, and synthetic data generation are used to overcome the challenges posed by low-resource settings. Models like multilingual BERT and mT5 are designed to learn from related languages, helping improve performance on low-resource tasks.
- **Impact:** This trend democratizes NLP, making it accessible and useful for more languages and communities worldwide, including those previously underrepresented in digital spaces.

2. Multilingual Models:

- **Definition:** Multilingual models are designed to understand and generate text in multiple languages without needing separate models for each language.
- **Examples:** Models like mBERT, XLM-R, and multilingual T5 are capable of handling numerous languages simultaneously. These models can perform tasks like translation, sentiment analysis, and question answering across different languages.
- **Advantages:** These models reduce the need for language-specific

training data and can leverage shared linguistic patterns between languages, making them highly efficient and versatile.

3. Zero-Shot Learning:

- **Definition:** Zero-shot learning allows models to perform tasks without having seen any specific training examples for those tasks. Instead, the models rely on general knowledge learned from related tasks or from a broad understanding of language.
- **Applications:** Zero-shot learning is particularly useful for tasks like text classification, where new classes can be introduced without additional training. It is also used in language translation, where a model can translate between language pairs it hasn't explicitly been trained on.
- **Future Potential:** This approach promises to significantly reduce the data requirements for deploying NLP models in new domains or languages.

Automated Text Generation:

1. Advancements in Generative Models:

- **Exploring Generative Models:** Recent advancements in generative models, such as GPT-3, ChatGPT, and other Transformer-based architectures, have revolutionized automated text generation. These models are capable of producing coherent, contextually relevant, and human-like text across various tasks, including conversation, content creation, and storytelling.
- **Capabilities:** These models can understand prompts, generate creative content, summarize long texts, and even assist in code generation. Their versatility has opened up new avenues in customer service, content creation, education, and entertainment.
- **Challenges and Considerations:** While generative models are powerful, they also pose challenges such as managing biases in generated content, ensuring factual accuracy, and handling ethical considerations around the use of generated text.

2. Real-World Applications:

- **Customer Support:** Generative models can automate responses in customer service, providing quick and personalized answers to common inquiries.
- **Creative Industries:** From generating marketing content to assisting with scriptwriting, these models enhance productivity and creativity in various fields.
- **Education:** Generative models can create personalized learning materials, quizzes, and explanations, making education more adaptive to individual needs.

Integration of NLP with Other AI Domains:

1. Combining NLP with Computer Vision:

- **Multimodal Models:** The integration of NLP with computer vision enables

the development of multimodal models that can understand and generate content across both text and image domains. Examples include OpenAI's CLIP and DALL-E models, which combine text and visual understanding.

- **Applications:** Such integration is crucial in fields like autonomous driving, where understanding road signs (text) in the context of the surrounding environment (images) is necessary. Another application is in the creation of alt-text for images, which enhances accessibility for visually impaired users.

2. NLP and Robotics:

- **Enhanced Interaction:** By integrating NLP with robotics, robots can understand and respond to natural language commands, enabling more intuitive human-robot interactions. This is particularly useful in assistive robots, home automation, and customer service robots.
- **Learning from Instructions:** Robots equipped with NLP capabilities can learn new tasks from textual instructions, enhancing their adaptability and usefulness in dynamic environments.

3. NLP and Other AI Domains:

- **AI in Healthcare:** NLP is used to extract meaningful insights from clinical notes, patient feedback, and research papers, while computer vision assists in interpreting medical images. Combining these allows for a more comprehensive analysis of patient data.
- **Financial Services:** In finance, NLP is used for sentiment analysis on market news, while machine learning models predict market trends. Together, these technologies enhance decision-making and strategy formulation.

Conclusion: The future of NLP is set to be shaped by advancements in emerging technologies, the rise of automated text generation, and the integration of NLP with other AI domains. These trends are driving the field towards more inclusive, efficient, and intelligent systems that can operate seamlessly across languages, modalities, and applications. As these technologies continue to evolve, they will unlock new possibilities and redefine what is achievable in the realm of human-computer interaction.

7. Summary

NLP deals with the interaction between computers and human (natural) languages. It encompasses a broad range of techniques and methods for processing, analyzing, and generating text data.

NLP is pivotal in extracting insights from unstructured text data, which constitutes a large portion of the data available today. Applications include sentiment analysis, machine translation, chatbots, and more.

The landscape of NLP is rapidly evolving, driven by innovations in model architectures, data availability, and computational power. As these trends continue, NLP will become even more integral to data science, enabling richer interactions between humans and machines. By mastering the foundational and advanced techniques outlined in this topic, data scientists can harness the full potential of NLP to solve complex problems and create impactful solutions.